Semana 4 · Interrupciones externas

Unidad 1 · sondeo, ISR, eventos, pulsadores y alerta doméstica

1. Sondeo vs interrupción

Sondeo (polling) -> el programa pregunta muchas veces si algo ocurrió. Se revisa en cada vuelta del ciclo principal.

↳ Con el sondeo el programa está preguntando una y otra vez por el estado del botón. Si demora mucho en dar la vuelta, puede tardar en detectar el cambio.

Sondeo -> ¿botón presionado? ¿y ahora? ¿y ahora?...

Interrupción -> una señal avisa al procesador que hay un evento para atender; no es necesario estar consultando todo el tiempo.

↳ Con la interrupción el ESP32 recibe un aviso al ocurrir el evento. Entonces ejecuta una rutina corta y después continúa con el trabajo que estaba haciendo.

Evento externo -> pin cambia -> se atiende interrupción -> vuelve el programa.

- Sondeo -> fácil, pero depende de qué tan seguido se revise el pin.
- Interrupción -> útil para reaccionar a cambios breves, sin revisar a cada instante.

Nota al margen: Interrupción no significa que todo se deba hacer dentro de la ISR...

2. Clases de interrupciones

Interna -> generada por un periférico interno, como un timer.

Externa -> evento que llega desde un pin (botón, sensor digital).

↳ La interrupción externa viene de una señal en un pin. Por ejemplo, si se abre una puerta y un sensor cambia de nivel, eso puede activar la alerta.

RISING -> flanco de subida, 0 a 1.

FALLING -> flanco de bajada, 1 a 0.

CHANGE -> cualquier cambio, 0 a 1 o 1 a 0.

Flanco -> instante en el que la señal cambia de nivel; no es lo mismo que mantener la entrada en HIGH o LOW.

Cuando se usa INPUT_PULLUP

Botón suelto -> HIGH. Botón presionado hacia GND -> LOW.

↳ Con pull-up el pin normalmente lee HIGH. Cuando presiono y conecto a GND pasa a LOW; ese cambio es FALLING. Depende de cómo esté armado el circuito...

Por eso suele interesar FALLING al presionar.

- Debe compartirse GND y elegirse un GPIO compatible; evitar pines con funciones especiales sin revisar el modelo.

3. ISR y bandera

ISR (Interrupt Service Routine) -> función que se ejecuta cuando se dispara la interrupción.

↳ No pondría toda la alarma dentro de la ISR: solo marco una bandera y salgo. Después el programa principal enciende el LED o el buzzer. Así la rutina dura poco.

ISR -> atender rápido -> marcar evento -> salir.

Flag o bandera -> variable que dice: “pasó algo”.

volatile -> avisa al compilador que esa variable puede cambiar fuera del flujo normal.

IRAM_ATTR -> atributo usado en ESP32 para ubicar la rutina en memoria de instrucciones RAM.

- Evitar en la ISR: delay, impresiones extensas y cálculos largos.
- Si la ISR comparte datos con el ciclo principal, conviene controlar el acceso; un contador compartido puede requerir protección.

```
volatile bool evento = false;

void IRAM_ATTR detectar() {
  evento = true;  // solo registrar
}
```

Nota al margen: La ISR no debería encargarse de toda la alarma; la tarea grande queda para el programa principal...

4. Rebote del pulsador (bounce)

Al presionar un botón mecánico, sus contactos pueden oscilar muy rápido antes de estabilizarse. Un toque puede parecer varios cambios.

Rebote -> varios flancos -> ISR se llama varias veces.

↳ Al presionar un botón puede haber varios cambios eléctricos muy rápidos, aunque yo lo haya presionado una sola vez. Por eso a veces parece que el contador suma más de uno.

Antirrebote -> filtro por tiempo o circuito físico.

- Si la señal tiene ruido, también puede aparecer una interrupción falsa.
- Idea: registrar el evento y verificar/filtrar el tiempo fuera de la ISR.

5. Ejemplo de estudio: alerta doméstica con ESP32

Montaje conceptual del tema de la semana: pulsador o sensor digital -> ESP32 -> LED de alerta. Se puede agregar buzzer si se dispone del componente adecuado.

Entrada -> GPIO 18 con pulsador a GND.

Salida -> GPIO 2 con LED y resistencia, si esa placa dispone de ese GPIO libre.

Evento -> FALLING -> bandera -> loop enciende la alerta.

Código base para comprender la secuencia

```
const int pinBoton = 18;
const int pinLed = 2;
volatile bool alertaPendiente = false;

void IRAM_ATTR avisoBoton() {
  alertaPendiente = true;
}

void setup() {
```

```
  pinMode(pinBoton, INPUT_PULLUP);
  pinMode(pinLed, OUTPUT);
  digitalWrite(pinLed, LOW);
  attachInterrupt(digitalPinToInterrupt(pinBoton),
                  avisoBoton, FALLING);
}

void loop() {
  bool activar = false;
  noInterrupts();
  if (alertaPendiente) {
    activar = true;
    alertaPendiente = false;
  }
  interrupts();

  if (activar) {
    digitalWrite(pinLed, HIGH);
    // alerta encendida; definir cómo reiniciar...
  }
}
```

- attachInterrupt -> asocia una función al cambio del pin seleccionado.
- noInterrupts() / interrupts() -> protegen brevemente la lectura y limpieza de la bandera.
- El código solo muestra el encendido de la alerta; falta elegir condición de apagado / reinicio y, si hace falta, antirrebote.

6. Revisar antes de probar

¿GND común? ¿botón bien conectado? ¿pin configurado?

Si se activa sola -> entrada flotante, ruido o rebote.

Si no se activa -> revisar tipo de flanco: RISING / FALLING.

Si se reinicia el ESP32 -> revisar conexiones, alimentación y trabajo excesivo dentro de rutinas.

Recordar -> polling pregunta; interrupción avisa; ISR atiende rápido; bandera marca evento.

Pendiente -> dibujar el circuito y decidir cómo se apaga la alerta...